

具身智能面试准备：数据预处理 / 格式 / 标注 / ROS2 回环

零声教育 Darren

只读这一份即可。结构：原理 → 白板手算 → 面试问答与经历模板。
深度：中高级；假定从零讲清 HDF5 / LeRobot / RLDS / Parquet / H.265。
先不懂「机器人到底在训什么」时：先读配套 [面试准备 机器人训练训什么.md](#)，再回来看格式选型。

目录

- 面试官视角（为什么问、想听什么、加减分、追问、评分条、不会时怎么接）
- 上篇 概念原理（HDF5 / RLDS / LeRobot / Parquet / 视频编码速览 / 清洗 / 标注 / ROS2）
- 中篇 白板手算（码率、GOP、连续窗、对齐、针孔像素）——数字唯一来源
- 下篇 面试问答（选型、坑、优化、场景题、经历模板）

怎么读：若缺「策略 π / IL vs RL」直觉，先看配套「训什么」；再上篇建格式概念（框架图用 Mermaid）；视频编码与对齐的手算只认中篇板书；下篇练口述。
同一结论若重复：面试答法以下篇为准；数字与公式以中篇为准；上篇第 4 章只是速览。

补充：面试官视角——他为什么问、想听什么、怎么答才加分

先看这节再背答案。下篇每道题的标准答法是「素材」；本节告诉你素材怎么用才像「真能干」。

核心原则：面试官不是在考你背书

面试官的底层逻辑只有一条：「这个人来了能不能干活、会不会踩坑、踩了坑能不能自己爬出来。」

所以他问每一个问题，背后都在评估三件事：

评估维度	他在想什么	你的回答要传递的信号
真做过 vs 背过	「他是真踩过坑，还是看了几篇博客？」	有具体数字、有「当时我……后来发现……」
工程判断力	「给他一个新场景，他能不能做取舍？」	不是「A 好 B 坏」，而是「在 X 条件下选 A，在 Y 条件下选 B」
失败处理	「出了问题他会不会慌、会不会甩锅？」	主动讲踩坑 → 定位 → 修复 → 防回归的闭环

再压缩成面试时的肌肉记忆：

- 结论（选型一句话）
- 原理（为什么）
 - 坑（我踩过/常见）
 - 怎么验（指标）
 - 如何回退（关特性 / 换缓存 / 降复杂度）

图 A · 面试口述五步



白板题：先写单位和假设，再算，最后说结论落在哪一层（归档 / 训练 / 部署）。数字错一点往往没事，口径乱才危险。

你以为在考 vs 他实际在筛

你以为在考	面试官实际在筛
HDF5 / LeRobot / RLDS 定义	会不会 分层：归档格式 vs 训练缓存
H.265 很强	知不知道 省盘 ≠ 省显存，随机采样何时会死
稀疏 / 稠密名词	会不会讲 主时钟、容差、valid、action 语义（注意：本文里不是「标注关键帧」那套含义）
YOLO / bbox	框是为 任务服务 还是为刷 AP
ROS2 同步	stamp 对不对、use_sim_time、安全门与回流

岗位不同，开口顺序不同

岗位	更盯你	建议开场
数据 / 平台	schema、校验闸门、压缩比、可复现转换	先讲转换流水线与质检
算法 (BC/VLA)	对齐相位、归一化、采样与 I0、分布偏移	先讲主时钟与训练视图
感知 / 标注	letterbox、伪标噪声、任务导向质检	先讲 AP ≠ success
ROS2 应用	stamp、QoS、安全仲裁、失败回流	先讲安全门与时间戳

对上面面试官时：先用他关心的那一层开口，再展开全链路，避免一上来背完整百科。

逐题拆解：面试官视角（对齐下篇题号）

□ Q1：HDF5、LeRobot、RLDS 怎么选？

面试官为什么问这个：

他不是想听你背表格。他想知道：你拿到一批新数据，第一步做什么决策？依据是什么？ 这直接决定你入

职后第一周能不能独立干活。

他想听到的信号：

- ☑ 你理解「归档」和「训练」是两个不同需求
- ☑ 你有「先问场景再选型」的习惯，而不是「我熟悉什么就用什么」
- ☑ 你知道选错了代价是什么（不是「不好」，是「具体会怎样」）

加分回答结构：

「我会先问三个问题：数据量多大、要不要强随机访问、团队用什么训练框架。
比如当时数据量不小、训练用 PyTorch、需要按时间窗采样——所以我 归档用 LeRobot (Parquet + MP4) 省盘分享，训练热集再转 HDF5/JPEG 缓存。
如果反过来只要 10 小时、团队跟 OpenX/TF——那导出 RLDS 对齐生态，没必要硬上三套。」

减分回答：

- × 「HDF5 适合结构化数据，LeRobot 适合视频。」（背书，没有决策过程）
- × 「我们用 LeRobot 因为它比较新。」（没有工程理由）
- × 「LeRobot 就是 H.265。」（格式是 MP4；编码常见 H.264，H.265 是可选）

常见追问：

追问	他想验证什么	你怎么接
「为什么不全库转 HDF5？」	成本分层意识	「全库原图/逐帧太贵；热集物化即可，冷数据留视频」
「v2 和 v3 差在哪？」	是否跟过生态坑	「v2 — episode — 文件会打爆 FS；v3 多 episode 拼 shard + meta offset」

□ Q3：Parquet「稀疏」和「稠密」怎么选？

面试官为什么问这个：

这是数据链路的核心概念。他想确认你理解的是 多模态时间对齐后的表结构，而不是把「稀疏」理解成「矩阵里很多零」，也不是把本题误答成「只标关键帧的稀疏标注」（那是另一套话题）。

他想听到的信号：

- ☑ 一句话本质：稀疏 = 各传感器原生频率分表保真；稠密 = 选定主时钟后每步凑齐一行
- ☑ 你知道代价：稀疏要 join；稠密可能插值造点、引入相位误差
- ☑ 你能说出主时钟、τ、valid

加分回答结构：

「采集我留稀疏真源：图 30Hz、关节 100Hz、动作 20Hz 各记各的。
训练前选主钟（策略频率或相机）做稠密物化；图像只能 nearest，关节可插值（注意角度 wrap），动作我倾向 20Hz。
超容差就 valid=0，绝不硬凑——否则 BC 会学错因果。」

减分回答：

- × 「稀疏就是很多零。」
- × 「我们对齐过了。」（说不出 τ / invalid / action 语义）

常见追问：

追问	他想验证什么	你怎么接
「误差多少算坏？」	任务相关判断	「理想上 $\leq$ 源半周期；工程用 $\tau + p95$ ；可换算成像素/厘米」
「ZOH 和最近邻差在哪？」	是否真想过标签定义	「两命令正中间 NN 会掷硬币；ZOH 取 $\leq t$ 的最后命令，更贴近控制语义」
「语言怎么办？」	schema 意识	「episode 级字段放 meta，或行内 broadcast / task_index」

□ Q6：数据清洗洗什么？

面试官为什么问这个：

这是区分「跑过 demo」和「真做过数据」的分水岭。没踩过脏数据坑的人，回答一定是泛泛的「去重、去异常」。

他想听到的信号：

- ☒ 你能列出 **具体失败模式**（时间/同步/控制/视觉/标签/分布）
- ☒ 你有「发现 → 定位 → 修复 → 防回归」闭环
- ☒ 你能说出 **数字或指标**（invalid rate、 $p95(\delta t)$ 、坏 episode 占比）

加分回答结构（STAR）：

- S：一批 episode 训练不稳 / loss 怪。
- T：排查数据质量。
- A：校验脚本查时间单调、帧数一致、对齐容差、黑帧、动作饱和、success 抽检……发现 stamp 回跳、图-动作错位、失败误标 success。
- R：剔除 + 分层（主集 vs recovery）后指标正常；脚本进入库流水线，新数据自动跑。

减分回答：

- × 「清洗就是去掉不好的数据。」
- × 「我用 pandas dropna。」（只碰到 NaN）
- × 把失败恢复段全删光还当优点（策略会变脆）

□ Q4：视频编码为什么省盘？代价是什么？

面试官为什么问这个：

他不一定要你当场推完 GOP 公式，但想确认你理解 **为什么不能长期堆 PNG/裸 RGB**，以及 **省盘会换来什么训练代价**。

他想听到的信号：

- ☒ 一句话本质：帧间冗余——相邻帧大量相似，视频主要存残差
- ☒ 代价：随机访问贵，平均要从附近 I 帧追到目标
- ☒ 关键纠正：省磁盘/带宽，不省 GPU 显存；连续窗与全局随机帧压力不同

加分回答（简洁版）：

「核心是帧间冗余。归档用 MP4（常见 H.264，更省可选 H.265）。
代价是随机抽帧 decode 贵；GOP 越长越省盘、随机越慢。
BC/ACT 连续窗往往可忍；世界模型强随机采样才逼我转 JPEG/HDF5 热缓存。
显存只看 decode 后的 $(B \times C \times T \times H \times W)$ ，和磁盘编码无关。」

数字去哪查： 中篇板书 A $(GB/h \approx 0.45R、(\mathbb{E}[N_{\text{dec}}]=(G+1)/2)$ 等)。

减分回答：

- × 「H.265 省显存。」
- × 「LeRobot 必须用 H.265。」

□ Q17：多传感器同步（ApproximateTime）

面试官为什么问这个：

ROS2 / 具身采集 **高频必问题**。他想知道你是不是真处理过「时间戳对不上」，而不是只会报包名。

他想听到的信号：

- ☒ 为什么不能 Exact：频率不同、时钟不同源
- ☒ ApproximateTime 的 **slop/容差** 含义与取舍
- ☒ 同步失败的后果要落到 **策略会学歪**，不是只说「数据不对」
- ☒ stamp 必须是采样时刻，不是回调 now()

加分回答结构：

「相机 30Hz、关节 100Hz、动作 20Hz——不可能戳完全相等。
在线用 ApproximateTime；离线可最近邻/插值 + 质检报告。
容差太大：图和动作不是同一瞬间，BC 相位错；太小：丢帧飙升。
我们用任务容忍定 τ ，看 $p95(\delta t)$ 和 invalid rate，而不是拍脑袋。」

常见追问：

追问	你怎么接
「某个传感器掉线？」	「在线同步可能卡住；要超时/降级标记 degraded，别静默用错配」
「 τ 怎么定？」	「先松后紧：看丢帧与任务像素/厘米误差，收到可接受区间」
「不用 message_filters？」	「离线 bag 转换更灵活；在线实时才强依赖 filters」

□ Q19：部署回环怎么安全接回 ROS2？

面试官为什么问这个：

区分「实验室选手」和「能上真机的人」。他想听安全意识，不是只听精度。

他想听到的信号：

- ☒ 第一反应是 **安全**，不是「把模型输出直接发给电机」
- ☒ 至少两道门：限幅 / watchdog / 仲裁（急停 > 遥控 > 策略）
- ☒ 训练-推理 **预处理一致**；失败自动录包回流

加分回答：

「部署和训练最大区别：训练数据错了最多 loss 高，部署错了可能撞坏东西。
三道门：动作反归一化后限幅；推理超时清零/安全姿态；急停与遥控优先于策略。
另外：同一份 preprocess；失败 trigger bag 回流清洗。」

减分回答：

- × 只谈「我们上了 TensorRT，延迟低」。
- × 忽略动作空间是绝对角还是增量速度。

□ Q10-Q16 / 伪标与检测（感知向）

面试官为什么问：

看你是刷 COCO AP，还是服务下游任务。

加分信号：letterbox 逆变换；伪标门控+时序平滑；**AP ≠ 成功率**；框抖先滤波/死区再上跟踪。

减分：「AP 到 0.x 就够了」；伪标无脑当 GT。

□ 中篇针孔 / 像素误差（白板追问）

面试官为什么问：

感知/标定方向会深问；数据工程方向多半只确认「你知道 3D→2D 会放大时间误差」。

最低限度可以说：

「 $u = f_x X/Z + c_x$ 。时间对齐误差 (Δt) 在速度 (V) 下变成 ($\Delta x_{px} \approx (f/Z)V\Delta t$)。
近距离 (Z 小) 更怕相位误差。我理解局限：理想针孔、实际有畸变要标定。」

- ☒ **策略**：结论和直觉要稳；公式推导记不清就诚实说物理含义，把数字指到中篇板书 B7。

□ Q22 : GPU 40%——如何证明是 decode ?

他想听： 对照实验，不是猜。

加分： time_to_batch vs forward；同一子集 MP4 vs JPEG/HDF5 缓存；利用率上去即 decode/I0 瓶颈。

万能应答框架（遇到不会的题）

不要沉默，不要瞎编：

第 1 句：承认边界

「这个具体实现我没有亲手在生产里做过。」

第 2 句：展示思维

「但我的拆法是……」（你会怎么查、怎么分层）

第 3 句：关联已知

「这和我做过的 XX 类似，当时我是……」

第 4 句：学习意愿

「如果入职需要，我可以按文档/最小实验在 X 天内补上。」

示例：

问：「用过 lifecycle node 吗？」

「生产里没用过（承认）。我理解它管启动顺序和状态，避免驱动没 ready 算法就在订空话题（理解）。我以前用 ready flag/手动门闩处理过类似问题（关联）。团队若规范用 lifecycle，看文档切过去很快（意愿）。」

面试官的评分心理

他问一个问题时，脑子里的评分条：

0分 —— 完全不知道 / 沉默

|

20分 —— 知道名词，说不清原理（「ApproximateTime 是做同步的」）

|

40分 —— 能讲清原理（「频率不同所以用时间窗匹配」）

|

60分 —— 原理 + 取舍（「 τ 是丢帧和相位误差的平衡」）

|

80分 —— 有数字 + 踩坑（「当时 τ 太大导致动作滞后，策略学歪了」）

|

100分 —— 踩坑 + 修复 + 防回归（「校验进 CI / 入库闸门」）

务实目标： 多数题稳定在 **40-60 分**；你真正做过的 1~2 个点冲到 **80**。不需要每题 100。
编造「百万小时量产」被拆穿，往往不如诚实的小闭环 + 清楚坑。

临场 30 秒自检（开口前过一遍）

1. 我有没有先给 **选型结论**？
2. 有没有说出 **条件**（什么情况下换方案）？
3. 有没有一个 **可测指标**？
4. 有没有 **回退/防回归**？
5. 数字说不清时，有没有改口成 **量级 + 假设**，而不是装精确？

上篇 · 概念原理

图 0 · 上篇一张图：数据工程在整条链上干什么



1. 先建立共同语言——一条机器人数据是什么

1.1 具身学习在学什么？

监督学习里，一张图对应一个标签；语言模型里，一段文本预测下一个 token。

具身 / 模仿学习学的是：在某个身体（臂、车、人形）上，

看到当前观测 → 输出下一步该怎么动

所以一条「有用的训练样本」通常不是一张图，而是：

在时刻 t ：

观测 `observation_t` （相机图像、关节角、夹爪开合、底盘速度、可选语言指令…）

动作 `action_t` （关节目标 / 增量、末端位姿、`cmd_vel`…）

（可选）`reward / success / 语言 / 检测框` …

然后进入 $t+1, t+2, \dots$

把从开始到结束的一整段连续时空数据叫一个 **episode（轨迹 / 回合）**。

一个数据集 = 很多 episode。

训练时 `DataLoader` 会从里面按策略取出 (`obs, action`) 或一段时间窗。

记住这句话：

格式之争（`HDF5` vs `LeRobot` vs `RLDS`）本质不是「文件后缀战争」，而是：

如何把「多模态、多频率、很长的时间序列」存到磁盘上，
让 采集便宜、分享方便、训练时读得又对又快。

1.2 一次采集里，各传感器频率为什么不同？

真实机器人（或仿真）里常见：

模态	典型频率	特点
RGB 相机	15–30 Hz	体积最大，延迟高
深度相机	15–30 Hz	大，且有无效深度
关节编码器 / 状态	50–200 Hz	小，必须准
控制指令 action	10–50 Hz（策略）或更高（底层）	和「学什么」强相关
语言指令	每个 episode 一句，或偶尔更新	几乎不是时间序列
检测框 / 语义	跟相机，或更低	可能稀疏（没目标就没有）

如果你强行要求「所有东西同一时刻各来一条、缺了就空着」，存储和训练代码会很痛苦——这就是后面 **稀疏 vs 稠密** 要解决的问题。

1.3 「落盘格式」和「训练 batch」不是同一层

很多人面试答混，先分清两层：

① 存储层（磁盘上长什么样）
rosvbag / mcap / HDF5 / Parquet+MP4 / RLDS(TFRecord) ...

② 训练视图（喂给 GPU 的张量长什么样）
固定 shape 的 batch：
images: [B, T, C, H, W]
state: [B, T, Ds]
action: [B, T, Da]

- 存储层可以 **稀疏、压缩、按文件分片**。
- 训练视图几乎永远要 **稠密、对齐、可向量化**。

所以工业界常见做法是：

采集/归档用一种省空间、保真的格式；
训练前再 materialize（物化）成好读的缓存。

图 B · 存储层 vs 训练视图（必须分层）

LeRobot 偏前者；HDF5 常做后者或传统全流程格式。

2. 三种主流机器人数据格式——从第一性原理讲

2.1 为什么需要「专用格式」？直接存一堆 PNG + CSV 不行吗？

可以，小项目完全行。规模一大就会痛：

1. **文件数量爆炸**：1 小时 × 30 FPS × 3 相机 ≈ 32 万张图；再乘 1000 小时，inode / 备份 / 列举目录都崩。
2. **对齐靠人脑**：frame_000123.png 和 joint.csv 第几行对应？差一帧全身错位。
3. **元数据丢失**：相机内参、fps、机器人型号、动作是绝对角还是增量、成功与否……散落在 README。
4. **训练 IO**：千万次 open() 小文件，比读几个大文件慢得多。
5. **生态**：别人的 ACT / Diffusion Policy / OpenVLA / LeRobot 训练脚本只认特定布局。

于是社区收敛出几套「约定」：HDF5、RLDS、LeRobot。

它们存的是同一类信息，差别在 布局、压缩策略、随机访问、和哪套训练代码绑定。

2.2 HDF5 到底是什么？（从「文件系统里的文件系统」讲起）

2.2.1 一句话定义

HDF5 (Hierarchical Data Format version 5) 是一种 二进制容器格式：

一个 .hdf5 / .h5 文件内部可以像文件夹一样有 Group (目录) 和 Dataset (多维数组)，还能挂 Attribute (元数据键值)。

你可以把它想象成：

```
my_dataset.hdf5      ← 一个真实磁盘文件
├── attrs: {fps: 30, robot: "aloha"}
├── episode_0/        ← Group
│   ├── observations/
│   │   ├── images/cam_high ← Dataset, shape (T, H, W, 3), uint8
│   │   └── qpos             ← Dataset, shape (T, 7), float32
│   ├── actions           ← Dataset, shape (T, 7)
│   └── attrs: {success: true}
├── episode_1/
└── ...
```

用 Python (h5py) 读某个关节轨迹，语义上像：

```
f["episode_0"]["observations"]["qpos"][100:120] # 只读 20 步，不必读整个文件
```

2.2.2 它为什么适合科学/机器人数据？

三个硬能力：

(1) 多维数组原生支持

图像就是 (T,H,W,C)，关节就是 (T,DoF)，不必自己设计二进制 layout。

(2) Chunked storage (分块存储) + 部分读取

数据集可以切成 chunk，例如按时间维每 64 帧一块。

你要读第 1000–1063 帧，磁盘只碰相关 chunk，这叫 **随机访问友好**。

(3) 过滤器流水线：压缩、校验

chunk 写出前可 gzip / lzf / szip 等压缩。

对图像也可存成「已经 JPEG 压缩的 bytes 数组」，或原始 uint8。

这就是 ALOHA、robomimic 等经典工作爱用 HDF5 的原因：**训练循环里疯狂随机抽时间片时，读得动**。

2.2.3 HDF5 的关键内部概念（面试能展开）

概念	含义	为什么重要
Group	树状命名空间	episode、camera 分层清晰
Dataset	同质多维数组	真正存数据
Attribute	小元数据	fps、成功标记、版本
Chunk	读写的原子块	决定随机访问粒度与压缩效率
Hyperslab	切片选择	data[t0:t1, ...] 高效
压缩 filter	按 chunk 压缩	省空间，但随机读要解压该 chunk

Chunk 怎么选（原理级）：

- chunk 太小：元数据开销大，顺序扫慢。
- chunk 太大：只想读 1 帧也要解压一大块。
- 机器人常见折中：沿时间维取几十到几百帧为一个 chunk，空间维整帧或整图。

2.2.4 HDF5 的致命缺点（必须会说）

1. **单文件容易变成几十~上百 GB 巨物**，复制、校验、云上部分拉取都难受。
2. **并发写不友好**：多进程同时写同一 .h5 容易锁死或损坏；采集端通常「一 episode 一文件」再合并。
3. **损坏局部修复难**：容器结构一坏，工具链不如「多个 Parquet/MP4」好抢救。
4. **分享与预览弱**：不像视频那样随手打开看；Hugging Face Hub 生态也不以 HDF5 为中心。
5. **图像若以原始 uint8 塞进 HDF5**：随机访问很快，但 **磁盘极大**；若在 HDF5 里存 JPEG bytes，访问变快仍在，但实现变复杂。

2.2.5 什么时候你应主动选 HDF5 ？

- 训练需要 **大量随机时间窗**（world model、某些 contrastive / JEPA、密集 BC 采样）。
- 已有代码栈（robomimic / 老 ACT 数据）只认 HDF5。
- 你愿意用磁盘换 decode 时间，或已经把图像降到 224 且可接受。

2.3 RLDS 是什么 ？

2.3.1 一句话定义

RLDS (Reinforcement Learning Datasets) 是 Google 一带推动的 「强化学习 / 机器人轨迹数据集」约定，通常落在 TensorFlow Datasets (TFDS) 体系里，底层常见是 TFRecord 分片。

它强调的是 语义：

```
Dataset
├── Episode (变长)
│   ├── Step (每一步一个嵌套字典)
│   │   ├── observation: {...}
│   │   ├── action: ...
│   │   ├── reward: ...
│   │   └── is_terminal / is_first / ...
```

Open X-Embodiment、RT-X 等大规模跨本体数据，很多是 RLDS/TFDS 这条线。

2.3.2 和 HDF5 的本质差别

	HDF5	RLDS
思想	「科学数组容器」	「RL episode/step 标准 schema」
底层	单个或少数 h5	TFRecord 等分片
随机访问	很强	较弱（顺序读友好，乱跳代价高）
生态	PyTorch 科研多	TensorFlow / JAX / OpenX 多

原理点： RLDS 赢在 **跨项目字段约定 + 大规模流水线**；不赢在「我随便 seek 第 937 个 episode 的第 55 帧图像」。

2.3.3 什么时候选 RLDS ？

- 你要跟 Open X-Embodiment / 某些 VLA 预训练数据对齐。
- 团队已是 TFDS 管线。
- 否则，新项目更常见 LeRobot 或 HDF5，需要时再 **转换导出** 一份 RLDS。

2.4 LeRobot 格式是什么？（Parquet + 视频 + Meta）

2.4.1 设计动机（先懂「为什么拆开」）

机器人数据里：

- **低维量**（状态、动作、时间戳）：小、结构化、要做过滤/统计/对齐 → 适合 **表（Parquet）**。
- **图像**：大、冗余高、适合用 **视频编码** 压 → 适合 **MP4**（社区常见 H.264；同画质更省盘可选 H.265）。
- **说明信息**（有哪些字段、shape、fps、每个 episode 多长）→ 适合 **JSON / 元数据表**。

于是 Hugging Face **LeRobot** 把数据集拆成：

```
dataset_root/  
  meta/          # 特征定义、统计量、episode 索引与边界  
  data/          # Parquet：每行一个 timestep 的低维字段  
  videos/        # MP4：按相机存的视频帧序列
```

训练时库会帮你：读到第 i 行 → 知道属于哪个 episode → 去对应视频里取第 k 帧 → 和 action 拼成样本。

2.4.2 Parquet 在这里扮演什么角色？

后面第 3 部分会细讲 Parquet。这里先记：

每一行 $\approx$ 对齐后的一个时间步（在「训练视图」意义上常是稠密的），
列包括：observation.state、action、timestamp、episode_index、frame_index、任务名等。

图像 **通常不进 Parquet 单元格** 当大 blob（理论上能，工程上差），而是：

行里有 frame_index，去 videos/cam_xxx/...mp4 解出那一帧。

2.4.3 v2 与 v3 的原理差异（面试加分）

v2 思路： 常常「一个 episode 一个 Parquet + 一个（每相机）MP4」。

- 好处：边界清晰、坏了只丢一段。
- 坏处：episode 上万时，**小文件海** 把文件系统、启动扫描、云对象列举拖死。

v3 思路： 多个 episode 拼进更大的 Parquet shard / MP4 shard，
用 meta/episodes 里的 **offset / length** 记录「第 N 个 episode 从文件哪里开始、多长」。

- 好处：文件少、初始化快、规模化。
- 代价：定位依赖元数据；工具要正确实现切片。

原理总结： LeRobot 把「表 + 视频 + 关系型元数据」当成可扩展数据湖的布局，而不是一个巨型数组文件。

2.4.4 LeRobot 的优势（对照 HDF5）

- 1. 体积：视频编码通常远小于原始帧塞 HDF5。
- 2. 可分享：Hub、版本、流式 StreamingDataset。
- 3. 可抽查：MP4 能直接打开看操作员演示是否离谱。
- 4. 与 HF 训练栈一体：记录、加载、训练脚本统一。
- 5. 列式分析：对 Parquet 用 DuckDB/Polars 做「成功率、动作范数、时长分布」秒级分析。

2.4.5 LeRobot 的代价

- 1. 随机抽帧贵：MP4 不是为「随机第 83721 帧」设计的（见视频编码一章）。
- 2. decode 吃 CPU/GPU：DataLoader 可能喂不饱训练。
- 3. 帧对齐必须严谨：视频帧数 $\neq$ 表行数就会静默错位——这是最致命工程坑。
- 4. 重随机访问任务 往往还要再转 HDF5/JPEG 缓存（格式不是宗教，是分层）。

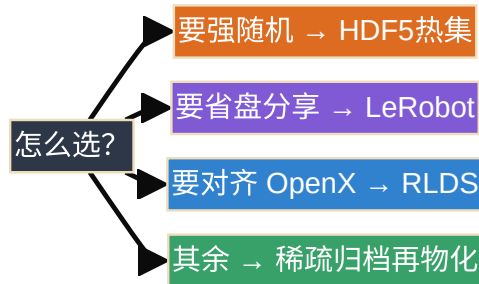
2.5 三式对比总表（原理视角）

维度	HDF5	RLDS	LeRobot
核心隐喻	层次化数组库	RL Episode/Step 标准	数据湖：表 + 视频
图像默认怎么存	数组或压缩 bytes 进文件	进 TFRecord	独立 MP4
低维怎么存	Dataset	Step 字典	Parquet 行
随机访问	强	弱-中	表（Parquet）强、视频弱（整体取决于 decode）
磁盘	原图很大；可压	中	通常更省
分享/生态	科研老栈	OpenX/TF	HF/LeRobot
典型痛点	巨文件、并发写	栈重、乱序采样	decode 与对齐

选型一句话（背这个）：

采集与社区分享优先 LeRobot；
跟 OpenX 对齐加 RLDS 导出；
训练若被视频随机解码卡住，对热数据做 HDF5/JPEG 物化缓存。
不要迷信单一格式贯穿采集到训练。

图 C · 三种格式怎么选（口述决策）



3. Parquet——以及「稀疏 vs 稠密」到底在说什么

3.1 Parquet 是什么？（列式存储）

3.1.1 行式 vs 列式

假设表有三列：timestamp, joint_0, rgb_path

行式（CSV/很多 OLTP）： 磁盘上大致

```
[t0, j0, path0][t1, j1, path1][t2, j2, path2]...
```

你只要统计 joint_0 的均值，也得扫过所有 path 字符串。

列式（Parquet）： 磁盘上大致

```
[t0,t1,t2,...] [j0,j1,j2,...] [path0,path1,path2,...]
```

只要读 joint_0 那一列，IO 少、压缩好（同类数据挨在一起，编码更狠）。

3.1.2 Parquet 还带什么？

- **Schema（强类型）**：float32 / int64 / list / struct...
- **Row Group**：文件内水平切段，便于并行与谓词下推。
- **列块压缩**：SNAPPY / ZSTD 等。
- **嵌套**：可以有 observation.state 这种结构体列（实现因版本而异）。

所以机器人用它存 **状态/动作/索引** 非常合适：要过滤「成功 episode」、算 action 分布，不必碰视频。

3.2 「稠密表」是什么意思？

稠密（dense）时间表 =：

规定一个 **主频率**（例如 20 Hz 或 30 Hz），
每一行都是一个完整时间步，该有的列都有值（或显式用 mask 标明无效）。

例子（主钟 = 相机 30 Hz）：

episode_index	frame_index	timestamp	state[7]	action[7]	lang
0	0	0.000	...	...	pick cup
0	1	0.033	...	...	pick cup
0	2	0.066	...	...	pick cup

注意：语言其实每个 episode 只说一次，但在稠密表里被 **broadcast（广播）** 到每一行——这是「用空间换训练简单」。

若关节原始是 100 Hz，你要放进 30 Hz 行，就必须：

- 下采样，或
- 插值 到相机时刻，或
- 存「最近邻关节」

这都会引入 **近似**。稠密的代价是：**可能造出物理上并未采样过的点**。

稠密的好处：

- PyTorch 里极易 stack 成张量。
- 不会出现「这个 batch 有的样本缺 action」的不规则结构。
- DataLoader 逻辑简单，bug 少。

3.3 「稀疏」是什么意思？

稀疏（sparse / ragged） = 按 **真实事件或原生频率** 存，不强行每一步填满所有模态。

例如三种表：

```
images.parquet : (episode_id, cam, stamp, video_frame_idx) @ 30Hz
joints.parquet : (episode_id, stamp, qpos[7]) @ 100Hz
language.parquet : (episode_id, text) @ 1 per episode
detections.parquet: (episode_id, stamp, label, cx,cy,w,h) @ 仅当检测到时
```

没有目标时，detections **根本没有行**——这才是「稀疏」。

稀疏的好处：

- **保真**：不插值、不伪造。
- **省空间**：空检测不占行。
- **扩展模态**：加一条触觉流，不必改宽表所有历史行。

稀疏的代价：

- 训练前必须做 **join + resample** 才能变 batch。

- join 错 stamp 会出幽灵 bug。
- 实现与版本管理更复杂。

3.4 「稀疏边、稠密中」——推荐你建立的架构心智模型

把两个词合成一句工程口号：

图 D · 稀疏边、稠密中、热缓存



面试时如果有人把「稀疏稠密」理解成「矩阵里很多零」，你可以纠正：

在具身数据语境里，稀疏/稠密首先指 时间对齐后的表是否强制每一步填满所有字段，以及 采集真源是否保留不规则采样；不是在说神经网络权重稀疏。

3.5 一个具体数值例子（对齐误差）

- 相机帧 stamp: 0.000, 0.033, 0.066
- 关节 stamp: 0.000, 0.010, 0.020, ...
- 策略要学的是：看到 0.033 的图时，人当时发的 action 是什么？

若 action 日志在 0.029 和 0.041：

- 最近邻可能选 0.029（误差 4ms）
- 或插值（若 action 是连续关节目标）

若你错误地把 0.033 的图配上 0.100 的 action（差 67ms），在快速运动里等于 学错因果。这就是为什么「稠密导出」必须有 最大对齐容差，超了标 invalid 或丢弃。

4. 图像为什么用视频编码——概念速览

手算、码率、GOP 期望解码次数、显存公式一律以中篇「板书 A」为准，本章只建心智模型，避免两处数字漂移。

口语里常说「省内存」，更精确是：省磁盘与传输带宽。解码后进 GPU 的张量显存与磁盘上用 H.264 还是 H.265 无关。

4.1 为什么不能堆原始 RGB？

[$H \times W \times 3$ \text{bytes}]

720p 单帧约 2.76 MB；三相机 × 30fps × 1h 到 ~0.9 TB 量级（细节见板书 A1）。采集必须压缩。

4.2 存图梯度（一句话版）

形态	删了什么冗余	随机访问	磁盘
PNG	无损，几乎不删	好	极大
JPEG 逐帧	主要空域	很好	中
H.264/H.265 MP4	空域 + 时域（帧间）	依赖 GOP，偏慢	小

- LeRobot 视觉侧是 MP4；社区常见 H.264，同画质更省盘可选 H.265（不是格式定义本身）。
- 视频省盘本质：背景几乎不动时，帧间残差很小。

4.3 I / P / B 与 GOP（只需记因果）

- I：可独立解码的入口；P：靠前面；B：双向，seek 更烦。
- GOP 越长：平均码率通常越低，但随机抽帧要从最近 I 帧追到目标 → 越慢。
- 连续时间窗会摊销 seek；全局随机单帧不会——这是 BC/ACT 与部分 world model 对格式压力不同的分岔（公式见板书 A5）。

图 E · 视频归档 vs 随机训练



4.4 工程结论（背这四句）

1. 归档用视频；训练若 GPU 空等，对热集做 JPEG/HDF5 解码缓存。
2. 省的是磁盘/带宽，不是显存。
3. CFR + 视频帧数与 Parquet 行数校验；错 1 帧 = 整段相位错。
4. 转码时尽量直接降到训练分辨率，避免每 epoch 解 720p/1080p 再 resize。

5. 数据预处理与清洗——在做什么、为什么必须做

5.1 「预处理」在具身链路里的四段

不要把预处理理解成「减均值」那么窄：

1. 采集时预处理：硬件同步、曝光、压缩、时间戳来源。
2. 离线同步对齐：多话题 → 统一时间轴。
3. 格式转换：bag → LeRobot / HDF5，写 schema 与 meta。

4. **训练时预处理**：resize、color jitter（慎用）、归一化、切片时间窗——必须与部署一致。

面试重点通常在 2 + 3 + 「清洗」；4 属于训练-部署一致性。

图 F · 预处理四段



5.2 时间同步原理（ROS2 也会问）

每个 message 应带 **stamp = 采样时刻**，不是回调执行的 now()。

ExactTime：时间戳完全相等才配对 → 真实异步传感器基本凑不齐。

ApproximateTime：在滑动窗口里找一组时间最近的消息，超过 slop 就丢。

对关节等高频低维：可用 **插值** 估某个图像时刻的状态。

对图像：不能「插出一帧新图」，只能选最近帧或丢掉。

仿真链路应统一 use_sim_time:=true（否则 bag 里 wall time 与仿真物理脱节）。若混入只打 wall time 的节点，要在采集规范里单独约定。

5.3 清洗在删什么？（按失败模式分类）

类别	例子	不洗的后果
时间坏	stamp 回退、大空洞	因果错乱
同步坏	图与关节差 100ms+	学错映射
控制坏	全程饱和、碰撞、急停	学危险动作
视觉坏	花屏、全黑、严重遮挡镜头	视觉特征垃圾
标签坏	失败当 success	策略乐观偏差
分布坏	同一条录 100 遍	过拟合假繁荣

高级点：清洗不是越干净越好。

「失败后恢复」段对鲁棒性有价值，应进 **单独子集** 或降权，而不是与完美演示无差别混训。

5.4 清洗要有指标，否则像玄学

可量化例子：

- 对齐误差 p50/p95
- 无效帧率、丢相机率
- action clip 比例
- 人工抽检的 success 标签准确率

- 场景/任务覆盖熵

有指标，你才能说「优化有效」。

6. 标注——BBox 与语义分割的原理

6.1 为什么模仿学习还要标注？

端到端 BC 可以不显式要框。标注仍然常见，因为：

1. **过滤**：目标不在视野的片段别当正样本。
2. **辅助监督**：检测/分割头让视觉表征更稳。
3. **语言落地**：把「红色杯子」对应到空间区域（grounding）。
4. **安全与部署**：在线检测做人接近、障碍。
5. **评测**：过程指标（是否对准）补充成败二元标签。

6.2 Bounding Box 在表示什么？

二维框是物体在图像上的 **轴对齐外接矩形** 近似：

常见参数化：

xyxy : (x_min, y_min, x_max, y_max) 像素

xywh : (x_min, y_min, w, h)

cxcywh: (cx, cy, w, h)

归一化 cxcywh: 以上再除以图像宽高 → 到 [0,1]

归一化的好处：分辨率变了，标签相对位置仍可解释；YOLO 系训练常用。

局限（原理）：框不知道形状、遮挡内部、深度；旋转物体浪费背景像素。

所以精细抓取有时要 **mask / 关键点 / 6D 位姿**。

6.3 检测算法在干什么？（不背全家桶，讲清楚流水线）

以现代 YOLO 类单阶段检测为例：

图 G · 检测流水线



Letterbox 为什么重要：

若直接拉伸resize，物体长宽比被拧巴，框与真实几何失真；letterbox 保比例。

部署必须与训练同一套 letterbox，且框要做逆变换——这是工程第一大坑。

6.4 目标检测 / 语义分割差在哪？

任务	输出	信息量
分类	整图一个类	最低
检测	多个框+类	哪里有什么
语义分割	每个像素一个类	区域形状，实例可黏连
实例分割	每个物体独立 mask	形状+实例分离
开放词汇检测/分割	文本查询 → 区域	不固定闭集类别

具身选型原理：

- 跟随、粗靠近、有无目标：检测框通常够。
- 可通行域、线缆、不规则接触：要分割。
- 「按自然语言指物」：要 grounding（开放词汇）。

6.5 伪标注（自动标注）原理

大规模 bag 不可能全人工：

帧 + 文本/类别

→ Grounding 模型出框（开放词汇检测）

→ （可选）SAM 根据框出 mask

→ 置信度门控 + 时序一致性（跨帧 IoU 匹配）

→ 抽检入库

噪声来源： 幻觉类、相似物混淆、时序跳变。

用法： 更适合 筛数据 / 辅助损失，不宜无脑当完美 GT 训安全关键模块。

6.6 从 2D 框到「位置估计」的误差链

若任务要 3D 位置：

2D 框中心（可能不是物理中心）

+ 深度图采样（透明/黑物失效）

+ 相机内参

+ 手眼/外参

+ 时间同步

→ 3D 点 / 粗框

任何一环的偏差都会放大。所以「标注算法估位置」在面试里要讲 **误差预算**，而不是只报一个检测 AP。

6.7 标注与控制耦合时的原则

检测框抖 → 控制抖：
优先 滤波、死区、降加速度、置信度门控；
再考虑跟踪器。
复杂层默认关闭、可回退——这是工程成熟度信号。

7. ROS2 采集同步与部署回环（原理补强）

7.1 采集侧

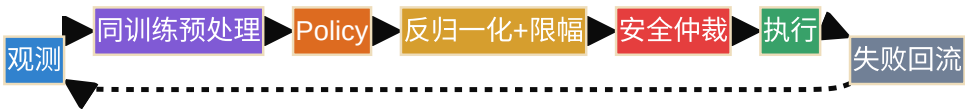
图 H1 · ROS2 采集



录制跟不上时：应 降采样/压缩，而不是把驱动堵死导致整车时间错乱。

7.2 部署回环

图 H2 · 部署安全回环



优先级：急停 > 遥控 > 策略。

最大原理坑： 训练与推理预处理不一致（颜色、letterbox、归一化 stats）。
离线 loss 很好、线上行为随机——优先查这一条。

8. 把原理收成「能对外讲的 3 分钟」

你可以这样讲（建议背熟结构，不要背全文）：

具身数据的本质是多模态时间序列。磁盘格式解决的是：怎么存得下、对得齐、读得动。
HDF5 是层次化数组容器，随机切片强，适合训练热缓存和老科研栈，但巨文件和图像膨胀明显。
RLDS 是 RL episode 语义标准，贴 OpenX。
LeRobot 把低维放 Parquet、图像放 MP4，用元数据维护 episode 边界，利于压缩、分享和分析。
稠密 是按主频率强制每步对齐，训练简单但可能插值造点；稀疏 保真原生频率，训练前再物化。
视频（H.264/H.265） 靠帧间预测大幅省存储，但随机解码贵，所以归档用 MP4、重随机训练再转

JPEG/HDF5。

清洗 抓时间、同步、控制、视觉、标签五类失败。

检测框/语义 不是装饰，用来过滤、辅助监督、语言落地和安全；框要统一坐标系与 letterbox 逆变换，伪标必须门控。

ROS2 负责正确时间戳与 Approximate 同步，部署要安全门和失败回流，飞轮才转得起来。

9. 对照检查清单（你是否真的懂了）

不用看文档，尝试回答：

1. 为什么说格式之争是「存储层 vs 训练视图」分层问题？
2. HDF5 的 chunk 如何影响「只读中间 64 帧」的性能？
3. LeRobot 为什么把图像从 Parquet 里拿出去？
4. 稠密表里语言列每一行都有，物理上发生了什么？
5. GOP 变长为什么省空间却拖慢随机训练？
6. 视频编码（H.264/H.265）省的是磁盘还是 GPU 显存？数字去哪章查？
7. 对齐容差过大/过小各会导致什么？
8. letterbox 不做逆变换，框会错在哪里？
9. 为何 AP 升高而抓取成功率不升？
10. 为什么失败恢复段不该无脑删光？

答不上来 → 回到对应章节重读；能答 → 再刷本文「面试问答」部分的场景题。

中篇·白板手算（视频编码 / GOP / 稀疏稠密对齐）

面试时只挑 2~3 个数字上板；下面是完整推导。

板书 A | 存储与视频编码（手算）

A0. 单位与口径（先统一，否则数字对不上）

存储口语常混用：

符号	含义	换算
1 KB	(10^3) B (硬盘厂商常用)	—
1 KiB	$(2^{10}=1024)$ B	—
1 MB	(10^6) B	—
1 MiB	$(2^{20}\approx 1.049\times 10^6)$ B	—
1 GB	(10^9) B	—
1 GiB	$(2^{30}\approx 1.074\times 10^9)$ B	—

面试约定（推荐说清楚）：

我下面用 **1 MB = 10^6 bytes**，和「码率 Mbps」一致；和 OS 显示的 GiB 会差约 7%。

码率：

$$[1\text{ Mbps} = 10^6\text{ bit/s} = 125\text{ KB/s} = 0.125\text{ MB/s}]$$

$$[\text{每小时体积(GB)} \approx \frac{\text{Mbps}}{8} \times 3600 / 1000 = 0.45 \times \text{Mbps}]$$

心算：码率 (R) Mbps → 约 (0.45R) GB/小时。
 例：8 Mbps → $(0.45\times 8=3.6)$ GB/h。

A1. 裸像素：RGB vs 相机常用 YUV420

RGB8 单帧

$$[S_{\text{RGB}} = H \times W \times 3]$$

分辨率	像素数	RGB 单帧	30fps 单相机 MB/s	1h GB ($\times 10^9$)
640×480	307,200	0.922 MB	27.6	99.4
1280×720	921,600	2.765 MB	82.9	298.6
1920×1080	2,073,600	6.221 MB	186.6	672
224×224 (训练)	50,176	0.151 MB	4.52	16.3

三相机 720p×100h：

$$[298.6 \times 3 \times 100 \approx 89,580\text{ GB} \approx \mathbf{89.6\text{ TB}}]$$

为何采集端常是 YUV420 不是 RGB？

YUV420：每 4 个像素共享一组 UV → 平均 **1.5 byte/像素**（不是 3）。

$$[S_{\text{420}} = H \times W \times 1.5]$$

720p 单帧 $\approx 1.382 \text{ MB}$ (约为 RGB 的一半)。
但这只是传感器/管道格式；训练进 PyTorch 通常仍转 RGB float，显存按 RGB/float 算。

板书记：

省采集带宽看 YUV；算训练显存看 decode 后的 RGB tensor。

A2. 用 bpp 精确化 JPEG 经验值

定义：

$$[\text{bpp} = \frac{\text{压缩后比特数}}{H \times W}]$$
$$[S_{\text{JPEG(MB)}} = \frac{\text{bpp} \times H \times W}{8 \times 10^6}]$$

720p (921,600 px)：

bpp	约等于质量直觉	单帧大小	30fps×1h
0.5	较狠	0.058 MB	6.2 GB
1.0	中等	0.115 MB	12.4 GB
1.5	常用演示	0.173 MB	18.7 GB
2.0	较保真	0.230 MB	24.9 GB

经验上常说「0.2 MB/帧」 $\approx \text{bpp} \approx 1.74$ ，落在合理区间。
面试应说：「我按 $\text{bpp} \approx 1.5 \sim 2$ 估 JPEG，720p 大约 0.17~0.23 MB/帧。」

相对裸 RGB (2.765 MB)：

$$[\frac{0.173}{2.765} \approx 6.3\%]$$

A3. 视频码率：把「几 GB」变成可调参数

公式

单相机每小时：

$$[\text{GB/h} = 0.45 \times R_{\text{Mbps}}]$$

三相机：

$$[\text{GB/h, total} = 0.45 \times R \times N_{\text{cam}}]$$

场景表（720p 操作演示，经验区间）

目标	每相机码率	GB/相机/h	3cam×100h
很省（可接受瑕疵）	2 Mbps	0.9 GB	270 GB
常用归档 H.265	4–6 Mbps	1.8–2.7 GB	0.54–0.81 TB
更保真 / H.264	8–12 Mbps	3.6–5.4 GB	1.1–1.6 TB
近视觉无损	20+ Mbps	9+ GB	2.7+ TB

同画质下 H.265 相对 H.264 少要码率 ~30–50%：
若 H.264 要 8 Mbps 才「够看」，H.265 常 4–5.5 Mbps 同级。

同码率下 H.265 通常更清晰——别把两种比较混为一谈。

和 JPEG 比（精细）

JPEG bpp1.5 → 18.7 GB/相机/h。
H.265 @ 5 Mbps → (0.45×5=2.25) GB/相机/h。

[$\frac{2.25}{18.7} \approx 12\%$]

即：相对 JPEG 再打到约 1/8（内容依赖；背景越静越夸张）。

A4. 编码控制：CRF vs CBR（面试加分）

模式	含义	对数据集
CBR	近似恒定码率	体积可预测；场景简单浪费、复杂处糊
VBR/CRF	按质量因子控质量	静段更小、动段更大；体积波动

具身演示：CRF + 上限码率 很常见。
关键：全库统一 CRF/preset/pix_fmt/fps，否则「同数据集画质不一致」污染训练。

转码时建议直接缩到训练分辨率（如短边 256/320）：

[$\text{像素比}_{720 \rightarrow 224} \approx \frac{224 \times 224}{1280 \times 720} \approx 0.054$]

码率可大幅下降；且 避免每个 epoch 解 720p 再 resize。

A5. I/P/B 与 GOP : 把模型做严谨

5.1 闭 GOP (Closed GOP) 简化模型

```
I P P P P P P P P I P ...
|←—— G 帧 ——→|
```

随机目标帧距离上一个 I 的偏移 ($D \sim \mathrm{Uniform}\{0,1,\ldots,G-1\}$)

无 B 帧时，必须解码帧数：

$$\lceil N_{\text{dec}}(D) \rceil = D + 1$$

$$\mathbb{E}[N_{\text{dec}}] = \frac{G+1}{2}$$

(口语常说 $(1+G/2)$ ，同一量级；更精确用 $((G+1)/2)$ 。)

G	$\mathbb{E}[N_{\text{dec}}]$
15	8
30	15.5
60	30.5
250	125.5

5.2 有 B 帧时发生什么？

显示顺序与解码顺序不一致，例如：

```
显示: I B B P B B P
解码: I P B B P B B    (示意)
```

后果：

- 1. seek 可能要解 **更多参考**；
- 2. 低延迟在线场景常关 B 帧；
- 3. 数据集随机访问更痛。

采集/训练数据集建议： $bf=0$ (无 B 帧) 或短 B 金字塔，换一点码率换 seek 确定性。

5.3 平均体积模型 (精细一点)

设 ($S_I = k S_P$)，GOP 内无 B：

$$\bar{S}(G) = \frac{k+(G-1)}{G} S_P$$

相对「纯 P 理想」的膨胀因子 ($\eta(G)=(k+G-1)/G$):

G	k=5 时 (η)	相对 G=60 的体积比
15	1.267	1.267/1.067 $\approx$ 1.19 (约 +19% 体积)
30	1.133	+6%
60	1.067	基准
120	1.033	-3%

洞察： 从 G=60→120，体积收益很小，但随机解码深度翻倍——**归档 GOP 别盲拉满。**

5.4 连续时间窗：复杂度被摊销（关键分岔）

策略训练常采样长度 (T) 的连续窗（如 (T=16)），而非 16 个全局随机帧。

若窗起点随机，先付一次 seek 代价 ($\approx D+1$)，之后每步 +1：

$$[N_{\text{dec,window}} \approx (D+1) + (T-1) = D + T]$$

$$[\mathbb{E}[N_{\text{dec,window}}] \approx \frac{G-1}{2} + T]$$

例：(G=60, T=16) → ($\mathbb{E} \approx 29.5+16=45.5$)

若 16 个**全局随机独立帧**：($\mathbb{E} \approx 16 \times 30.5 \approx 488$)

差一个数量级！

所以：

同一套 H.265, ACT/Diffusion **连续窗** 往往可忍；
世界模型随机时间对 / 强打乱帧 才逼你转 JPEG/HDF5。

这是中高级必须会的分岔。

5.5 Batch 级工作量

设 batch 内有 (B) 个独立窗（不同 episode）：

$$[W \approx B \times \left(\frac{G-1}{2} + T \right)]$$

设定	W 量级
B=64, T=16, G=30	64 × (14.5+16) $\approx$ 1952
B=64, T=16, G=60	64 × (29.5+16) $\approx$ 2912
B=64, T=1, G=60（等价随机单帧）	64 × 30.5 $\approx$ 1952
B=64, T=1, G=250	64 × 125.5 $\approx$ 8032

再乘 **相机数**（三相机各解一遍）：×3。

A6. 「不省显存」——把账算到 tensor

假设 decode 后：float32、RGB、训练分辨率 (H=W=224)、时间长 (T)、batch (B)、相机 (C)。

$$[\text{Bytes}] = B \times C \times T \times 3 \times H \times W \times 4$$

例：(B=64, C=3, T=16, 224^2):

$$[64 \times 3 \times 16 \times 3 \times 224 \times 224 \times 4]$$

先算 ($224^2=50176$)

$$[3 \times 50176 \times 4 = 602112 \times \text{(单帧单相机 float RGB)}]$$

$$[64 \times 3 \times 16 \times 602112 \approx 1.85 \times 10^9 \times \text{B} \approx \mathbf{1.85 \text{ GB}}]$$

仅观测图像就 ~1.9GB 量级 (还未算激活、状态、优化器)。

与磁盘是 H.264 还是 H.265 无关。

若误用 720p float 进 GPU:

$$[\frac{1280 \times 720}{224 \times 224} \times 18.4 \times \rightarrow \sim 34 \times \text{GB}]$$

直接爆。所以：**转码时降分辨率是显存与 decode 双重优化。**

A7. Decode 吞吐与 GPU 利用率 (可叙述的 profiling 逻辑)

定义:

$$[\eta_{\text{GPU}} \approx \frac{T_{\text{compute}}}{T_{\text{compute}} + T_{\text{wait IO/decode}}]$$

若一步 compute=20ms, wait=30ms $\rightarrow (\eta=40\%)$ 。

定位实验 (面试可讲):

1. 同一子集, MP4 随机窗 vs 预解码 JPEG/HDF5;
 2. 若 (η) 从 40% $\rightarrow$ 80%+, 即 decode 瓶颈;
 3. 再试: 短 GOP / 连续窗 / NVDEC / 降分辨率转码。
-

A8. 帧索引错位: 最致命的精细坑

设 Parquet 行 frame_index = 0..N-1, 视频也有 N 帧。

若转码多写了 1 个重复首帧, 或 OpenCV 与 PyAV 对「最后一帧」处理不同:

$$[\text{错位量} = \Delta \in \{+1, -1\}]$$

则整 episode 的 action 相对图像平移 (Δ) 步。
在 30Hz 下 ($\Delta=1$) $\rightarrow$ **33ms 系统性相位误差** (见板书 B 的危害)。

强制校验：

```
assert abs(n_frames_video - n_rows) <= 0
# 或容差 0，失败则重编码/丢弃 episode
```

抽检：固定 seed 解第 0/中/末帧，与 PNG 黄金样本 hash 比。

A9. 板书 A 终局决策

归档全库

- └ H.265/H.264 + CFR + 统一 CRF + (建议) bf=0
 - + 最好已是训练分辨率

训练采样形态？

- └ 连续窗为主 (BC/ACT/DP)
 - | └ 预取顺序 decode 通常可接受；GOP 30-60
- └ 强随机单帧/随机对 (部分 WM)
 - └ 热集 JPEG/HDF5/LMDB 物化

永远：

- 帧数校验、CFR、颜色空间固定、显存按 decode 后分辨率算

90 秒口述

「裸 720p RGB 约 300GB/相机/小时。JPEG 按 bpp1.5 大约 19GB。视频我按 Mbps 算：GB/h $\approx$ 0.45R，H.265 常用 4-6Mbps 就是 2-3GB/相机/小时。省的是帧间残差。随机访问成本期望约 (G+1)/2 帧；但连续窗是 D+T，不是 T 次独立 seek——所以 BC 连续窗和世界模型随机采样对格式压力完全不同。显存只看 $B \times C \times T \times 3 \times H \times W \times 4$ ，和 H.265 无关。因此归档视频、热点物化、转码直接降到训练分辨率。」

板书 B | 稀疏稠密对齐 (手算)

B0. 先定语义：你对齐的是哪一种「动作」？

面试里 80% 的对齐争论其实是 **标签定义不清**：

符号	含义	常见用途
(a_t^{cmd})	在时刻 t 发出 的命令	学「看见就发令」
(a_t^{exec})	在时刻 t 执行器侧生效 的指令	含通信/控制延迟
(a_t^{Δ})	相对增量（如 (Δq) ）	Diffusion/部分 BC
(s_t)	状态（qpos 等）	观测的一部分
(o_t)	图像观测	不可插值

模仿学习最常见监督：

$$[\text{学}] \pi(o_t, s_t) \approx a_t^{\text{cmd}}]$$

若误把「延迟后的 exec」标成与 (o_t) 同时，等于把 **系统延迟揉进策略**，真机延迟一变就崩。

meta 必须写死：action 是 cmd 还是 measured，绝对还是 delta，单位，限幅。

B1. 稀疏真源的形式化

每条流 (m) 是时间有序集合：

$$[\mathcal{S}_m = \{(t_i^{(m)}, x_i^{(m)})\}_{i=1}^{N_m}, \quad t_{i+1} \geq t_i]$$

稀疏 = 各 $(\mathcal{S}_m)$ **独立存储**，不要求 (t) 相等。

5.0s 例子（理想无丢包）：

流	f	N
img	30	150
q	100	500
a	20	100
lang	—	1

若图像实际丢了 3%：

$$[N_{\text{img}} \approx 150 \times 0.97 = 145.5 \rightarrow 145 \sim 146]$$

稠密导出不能假装还有 150 个完美点——要用 **valid** 或重采样策略。

B2. 稠密物化的形式化

选主时钟 $(\mathcal{T} = \{t_j\}_{j=1}^N)$ ，常取：

- 某相机所有帧时间戳，或
- 均匀格点 $(t_j = t_0 + j/f_m)$

对每个 (t_j) 、每个模态 (m) ，定义对齐算子 $(\mathcal{A}_m(t_j; \mathcal{S}_m) \rightarrow (\hat{x}, \delta t, \text{ok}))$ 。

稠密第 j 行：

$$[\text{row}]_j = \big[\hat{o}, \hat{s}, \hat{a}, \text{lang}, \delta t_o, \delta t_s, \delta t_a, \text{valid}]_j, \big]$$

$$[\text{valid}]_j = \bigwedge_m \text{ok}_m \wedge \text{其他业务规则}$$

B3. 最近邻与误差：精确写法

最近邻：

$$[i^{\star}(t) = \arg \min_i |t_i - t|, \quad \delta t = t_{i^{\star}} - t, \quad \hat{x} = x_{i^{\star}}]$$

在理想均匀采样、无抖动时：

$$[|\delta t| \leq \frac{1}{2f} = \frac{T}{2}]$$

有抖动时上界失效。 设采样间隔为 $(T + \epsilon_i)$ ， (ϵ) 为抖动，则可能出现更大空隙。故工程不用理论 $(T/2)$ 当唯一门控，而用：

$$[\text{ok} \iff |\delta t| \leq \tau_m]$$

(τ_m) 按任务选（见 B8），并统计 $p_{95}(|\delta t|)$ 。

B4. 线性插值（状态）与角度 wrap

对标量/向量状态，找到 $(t_i \leq t \leq t_{i+1})$ ：

$$[\alpha = \frac{t - t_i}{t_{i+1} - t_i}, \quad \hat{x} = (1 - \alpha)x_i + \alpha x_{i+1}]$$

关节角 wrap（关键精细点）：

若 $(x_i = \pi - 0.05)$ ， $(x_{i+1} = -\pi + 0.05)$ ，朴素插值会穿过 0 走错方向。应在差分时报 wrap 到 $(-\pi, \pi]$ 再插。

伪代码：

```
d = wrap(x1 - x0) # to (-pi, pi]
x = wrap(x0 + alpha * d)
```

速度信号可插，但噪声放大；夹爪离散位应用 **阶跃保持（ZOH）** 而非线性插。

图像：**禁止插值当 GT**；只能 nearest，或标 missing。

B5. 因果动作对齐的两种政策

政策 NN（最近邻）

实现简单；在（t）落在两个 cmd 正中间时歧义最大。

政策 ZOH / 「上一个命令仍生效」（常更物理）

$$[\ \hat{a}(t) = a_{i}\ \text{where}\ t_i=\max\{t_k: t_k\leq t\} \]$$

即：命令在下一次更新前保持。

这与许多实时控制语义一致。

例：cmds 在 0.975→A，1.025→B。

问 t=1.000：

- NN：A/B 间距 25ms，歧义
- ZOH：选 A（仍生效）
- 「下一命令」前瞻：选 B（泄露未来，一般不当作人类演示标签）

学遥操作演示时，优先 ZOH 或「命令时刻 ≤ 图像时刻的最后一条」。

B6. 完整时间戳对照表（请在纸上抄一遍）

稀疏真源（节选）

actions (20Hz 理想)：

0.900:A0, 0.950:A1, 0.975 实际抖动→记 0.975:A2, 1.025:A3, 1.050:A4

为贴近旧例，用：

t_a	action
0.950	A1
0.975	A2
1.025	A3
1.050	A4

images (30Hz)：

t_o	frame
0.933	28
0.967	29
1.000	30
1.033	31
1.067	32

qpos (100Hz) 片段： 0.980, 0.990, 1.000, 1.010, 1.020, 1.030, 1.040...

稠密导出参数

- master = 相机时间戳
- qpos: 线性插值, ($\tau_q=8$) ms (很严, 因 100Hz)
- action: ZOH ($\leq t$ 的最后命令), ($\tau_a=30$) ms (允许略大于 25ms 理论半周期, 因有抖动)
- 若 ($|t-t_{\text{cmd}}| > \tau_a$) 且距离下一条也太远 $\rightarrow$ invalid

结果表

t	frame	qpos 来源	action 政策	δt_a	valid	说明
1.000	30	恰有 1.000 样本 / 或插值	ZOH $\rightarrow$ A2@0.975	-25ms	1 (若 $\tau=30$)	NN 会在 A2/A3 间打架
1.000	30	同上	NN, $\tau=20$	25ms	0	对比: 更严容差
1.033	31	插 1.030-1.040	ZOH $\rightarrow$ A3@1.025	-8ms	1	干净
1.033	31	同上	NN $\rightarrow$ A3	-8ms	1	与 ZOH 一致

面试价值： 同一数据, $NN+\tau=20$ 与 $ZOH+\tau=30$ 对 $t=1.000$ 的 valid 不同——说明对齐是 带政策的算法, 不是「对一下时间戳」四个字。

B7. 针孔几何：把秒误差变成像素误差

针孔近似 (小角度):

$$[x_{\text{px}}] = \frac{f_{\text{px}}}{Z} \cdot X + c_x$$

水平速度 (V_X) 引起的像素速度:

$$\dot{x}_{\text{px}} \approx \frac{f_{\text{px}}}{Z} V_X$$

时间误差 (Δt) $\rightarrow$ 像素误差:

$$\Delta x_{\text{px}} \approx \frac{f_{\text{px}}}{Z} V_X \Delta t$$

代入可辩护数字

设：

- $(f_{\text{px}}=600)$ px (约 $\sim 50^\circ$ 水平 FOV @ 1280 宽的粗量级； 70° 时 (f) 更接近 $\sim 900\text{px}$)
- $(Z=0.6)$ m
- $(V_X=0.4)$ m/s
- $(\Delta t=0.05)$ s

$$[\Delta x_{\text{px}} \approx \frac{600}{0.6} \times 0.4 \times 0.05 = 1000 \times 0.02 = \mathbf{20 \text{ px}}]$$

对应位移 2cm：

$$[\Delta X = V_X \Delta t = 0.02 \text{ m}]$$

$$[\Delta x_{\text{px}} \approx \frac{600}{0.6} \times 0.02 = 20 \text{ px}]$$

一致。

若 $(Z=0.3)$ m (更近)：像素误差 **翻倍到 40px**——近距离操作更怕相位误差。

相对图像宽度

720p 宽 1280: $20\text{px} \approx 1.6\%$ 图宽。

对「杯口对准」类任务，这已可观；对「房间导航跟随」可能仍可忍。

→ 容差必须任务相关。

B8. 容差 (τ) 怎么选？（精细经验规则）

经验不等式：

$$[\tau_m \lesssim \min\left(\frac{1}{2f_s}, \frac{\Delta x_{\text{tol}}}{V_{\text{typ}} \cdot (f_{\text{px}}/Z)}, T_{\text{control}}\right)]$$

其中 (f_s) 是**被对齐的源频率**（如 action 20Hz），不是主时钟频率。

口述版：

1. 不大于源半周期太多（否则失去「近邻」意义）；
2. 换算到像素/米后不超过任务公差；
3. 不超过控制周期太多。

任务	建议图像-动作 τ	慢速桌面 pick-place	20–40 ms
中速跟随/移动操作	10–25 ms	高速接触/拍击	≤ 10 ms + 硬件同步

B9. 多相机稠密：另一个精细坑

三相机曝光不同步时：

cam	stamp
head	1.0000
left	1.0040
right	0.9975

若强行同一行「假装同时」，立体/跨视几何会歪。

策略：

1. 选 master cam；其它 cam nearest，写 (δt_{cam}) ；
2. $(\max|\delta t_{\text{cam}}| > \tau_{\text{stereo}})$ 则该行不用于需要立体的损失；
3. 仍可用于「单眼前视」策略头。

B10. 稠密导出伪代码（可默写结构）

```
function export_dense(episode, f_m or master_stamps, policies, taus):
    rows = []
    for t in master_timeline:
        o, dt_o, ok_o = nearest(images, t, tau_o)      # 图像不可插
        s, dt_s, ok_s = interpolate(qpos, t, tau_s)    # wrap angles
        a, dt_a, ok_a = zoh_or_nn(actions, t, tau_a)
        lang = episode.language                        # broadcast
        valid = ok_o and ok_s and ok_a and business_rules(...)
        rows.append({t, o_ref, s, a, lang, dts..., valid})
    verify_video_frame_count(rows)
    write_parquet(rows); encode_video(...)
    write_meta({master, policies, taus, fps, action_def, git_hash})
    return quality_report(rows) # p50/p95 dt, invalid rate
```

B11. 质检指标（没有指标等于没做清洗）

对每个 episode / 全库：

指标	健康方向
invalid 行比例	低；突然升高说明时钟崩了
$ \delta t_a $ 的 p50/p95	$p95 < \tau$
大间隙 (>3T) 次数	近 0
stamp 非单调次数	0
视频帧数 $\neq$ 有效行数	0
动作饱和比例	监控，过高要审演示质量
成功标签抽检准确率	高

B12. ROS2 ApproximateTime 与离线对齐的关系

在线 `message_filters::ApproximateTime`：

- 每流一个队列；
- 取时间戳「足够近」的一组；
- 超差丢弃。

离线 bag 转换 = 同一思想的批处理版，但你可以：

- 双向看未来（离线允许）；
- 对状态插值；
- 打出完整质检报告；
- 用 ZOH 政策标 action。

注意：在线同步丢的数据，若没进 bag，离线救不回——采集质量是上限。

B13. 90 秒口述

「稀疏是各模态原生时间序列；稠密是选定主钟后的物化表。图像只能 nearest，关节可 wrap 插值，动作我倾向 ZOH：取 $\leq t$ 的最后命令，避免 NN 在两个 cmd 正中间掷硬币。容差超了就 valid=0。50ms 相位误差在 $f=600\text{px}$ 、 $Z=0.6\text{m}$ 、 0.4m/s 时大约 20 像素，近距离翻倍。所以对齐政策、容差、action 语义都要写进 meta，并且看 $p95(\delta t)$ 和 invalid rate，而不是说一句‘我们同步过了’。」

板书 C | 串烧追问

C1. 为何不全库 HDF5 原图？

分层账：

层	形态	量级（3cam×100h，720p 演示）
裸 RGB	不可接受	~90 TB
JPEG 逐帧	可随机访问	~5–7 TB
H.265 归档	默认可分享	~0.5–1.5 TB
训练热集 224 JPEG/HDF5	当前实验子集	随子集变化，常 << 全库

随机访问需求用 **子集物化** 满足，不把全库永久留在最贵形态。

C2. 稀疏如何进 DataLoader？

两种实现，语义相同：

1. **离线稠密物化**（推荐可复现）；
2. **在线**：sampler 输出 (ep, t) → 各模态 gather → **堆成稠密 batch 张量**。

不会出现「一个 batch 里有的样本缺 action 列」还硬 stack——缺的在 gather 前就 mask/重采样丢掉。

C3. 连续窗能否容忍更长 GOP？

能。复杂度 ($\approx B(C)(\frac{G-1}{2}+T)$)，T 增大时 seek 占比下降。
但 **首帧延迟与多相机** 仍在；要 profiling，不靠猜。

C4. 仿真 GT 与真机对齐差异？

仿真可完美同步；真机有暴露不同步、USB 调度、无线遥操作延迟。
用仿真数据训的对齐假设迁到真机必挂—— **τ 与政策按真机重测**。

临场默写纸（半页）

单位：GB/h $\approx 0.45 \times$ Mbps
720p RGB ≈ 2.765 MB/帧 $\rightarrow \approx 299$ GB/相机/h $\rightarrow \times 3 \times 100h \approx 90$ TB
JPEG bpp1.5 $\rightarrow \approx 0.173$ MB/帧 $\rightarrow \approx 19$ GB/相机/h
H.265 @5Mbps $\rightarrow 2.25$ GB/相机/h

$E[N_{dec}] = (G+1)/2$ 随机单帧
 $E[N_{win}] = (G-1)/2 + T$ 连续窗（关键分岔）
显存 $\approx B \cdot C \cdot T \cdot 3 \cdot H \cdot W \cdot 4$ 224³例 B64 C3 T16 ≈ 1.85 GB

稠密：master + A_m + valid
图像 NN；关节插值+wrap；动作 ZOH 优先
e 理想 $\leq T/2$ ；工程用 $\tau + p_{95}$

$\Delta x_{px} \approx (f/Z) \cdot V \cdot \Delta t$ 例 $600/0.6 \cdot 0.4 \cdot 0.05=20px$
帧数 assert; CFR; bf=0; 转码到训练分辨率

自测

1. 为什么说「同画质少 40% 码率」和「同码率更清晰」不能混用？
2. 推出 $GB/h=0.45R$ 。
3. 连续窗与随机单帧的期望解码次数公式差在哪一项？
4. ZOH 与 NN 在两命令中点为何分歧？
5. 用针孔公式重算 $Z=0.3m$ 时的像素误差。
6. 为何转码到 224 既省盘又省显存又省 decode？
7. valid 与 τ 该进 meta 的哪些字段？

下篇 · 面试问答（中高级）

原理能讲通后再背「选型 + 坑 + 优化」。经历模板在文末，换成你真实做过的子集。

答题前默念：结论 → 原理 → 坑 → 指标 → 回退。逐题「他为什么问」见文首「面试官视角」。

0. 开场总览（必答）

Q0. 用 2 分钟讲一条「采集 → 训练 → 部署」闭环

答：

具身数据链路可以压成六层：

1. **采集**：ROS2 多话题（图像、关节/底盘状态、`/cmd_vel` 或关节指令、TF、可选语言指令）。
2. **同步对齐**：按主时钟（通常控制频率或相机）做 `associate` / 插值，丢掉不可恢复的坏段。
3. **落盘**：短期用 `rosbag2/mcap`；训练侧转成 **HDF5**（随机访问友好）或 **LeRobot (Parquet 表 + MP4 视频；编码多为 H.264，也可 H.265)**。
4. **清洗与质检**：失败 episode、人手干预、时间戳乱序、动作饱和、画面过曝等过滤；产出质量报表。
5. **标注 / 伪标注**：`bbox`、语义 `mask`、任务成功标签、语言对齐；用于辅助损失或检索式数据筛选。
6. **训练与回灌**：策略/VLA 训练 → ONNX/TensorRT 或原生 PyTorch 推理节点 → 安全层仲裁后下发；线上失败回流再清洗。

中高级面试官要听的不是名词，而是：**哪一层最脏、哪一层最贵、你怎么用指标证明改对了**。

追问答法：

- 最脏：时间同步 + 失败轨迹混入成功集。
- 最贵：高分辨率多相机原始图落盘与训练 decode。
- 证明：磁盘占用、DataLoader GPU 空等、策略成功率、坏 episode 占比。

1. 数据格式与预处理

Q1. HDF5、LeRobot、RLDS 怎么选？各自优势是什么？

答（选型结论先说）：

格式	形态	优势	劣势	典型场景
HDF5	层次化二进制，episode/group + dataset	随机访问极强、chunk+压缩成熟、科研栈（ALOHA/robomimic）通用	单文件易巨大、并发写锁、Hub 分享弱、损坏难局部修复	需要高频随机抽时间窗（world model、BC 密集采样）
LeRobot	Parquet（状态/动作）+ MP4（视觉）+ meta	Hub 分享、流式读取、可视化友好、生态增长快	视频随机 seek/decode 贵；schema 演进要跟版本（v2→v3）	模仿学习、社区数据集、快速迭代
RLDS	TFDS/TFRecord 风格 episode 序列	Open X-Embodiment / RT-X 互通	TF 栈重、随机访问弱于 HDF5	跨本体大规模混训

选型原则：

- 训练代码读什么，就主存什么；格式是契约不是信仰。
- 分享与采集默认 LeRobot；重随机访问训练可二次导出 HDF5；要对齐 OpenX 再出 RLDS。
- 生产上常见：采集 LeRobot / bag → 训练前按任务转一份训练友好缓存（HDF5/Zarr）。

坑：

- 只存一种格式，换框架就全量重转，且内外参/action 定义在转换中悄悄丢。
- LeRobot v2 「一 episode 一文件」到万级 episode 会打爆文件系统；v3 改成多 episode 拼 shard + meta 记录 offset。

解决：

- 转换管道强制 schema 校验（shape/dtype/单位/相机名）。
- 大库跟 v3：少文件、靠 meta/episodes 定位边界。

优化：

- 热数据（本周训练）落本地 SSD 的 HDF5/解码后 cache；冷数据留视频+Parquet。
- 转换只做一次，训练读 cache；不要每个 epoch 从 MP4 全量硬解。

Q2. 为什么工业界倾向「Parquet 元数据/低维量 + 视频存图」，而不是全塞进一个大 HDF5？

答：

- **容量**：RGB 是数据主体。H.264/H.265 压缩比通常远高于 HDF5 里 JPEG/gzip 逐帧存；同样 100h 多相机数据，视频方案磁盘可差一个数量级。
- **可读与协作**：Parquet 可用 DuckDB/Polars 秒查「哪些 episode 成功、action 范数分布」；视频可用普通播放器抽查。
- **流式与云**：Hub/对象存储对「大文件少个数」更友好；整库 HDF5 往往要整文件拉完。
- **职责分离**：低维高频信号要精确、可列式过滤；图像要压缩与顺序解码——两种物理特性不该共用一种最优存储。

坑：以为「分开存」就永远更好。World model / 强随机时间窗采样时，MP4 seek 会把 GPU 喂饿。

解决：训练前做 **decode cache** (LMDB/HDF5/WebDataset/本地 raw tensor shard)，或限制采样策略（同 episode 连续窗 + 预取）。

Q3. Parquet「稀疏」和「稠密」在机器人数据里分别指什么？怎么选？

答：

这里的稀疏/稠密不是矩阵理论口号，而是 **schema 与落盘子策略**：

稠密 (dense table)

- 每一行 = 一个对齐好的 timestep。
- 列固定：observation.state, action, timestamp, episode_index, frame_index, 可选 language。
- 缺测必须 **插值 / 前向填充 / 显式 mask 列**，否则行对不齐。
- 优点：DataLoader 简单、向量化快、训练代码干净。
- 缺点：传感器频率不一会制造假数据；padding 浪费；强制对齐可能引入相位误差。

稀疏 (sparse / ragged)

- 按话题或事件存：图像 30Hz、关节 100Hz、语言一条/episode、检测框只在有目标时出现。
- 用 offset、变长 list、或分表 (images.parquet / proprio.parquet) + join key。
- 优点：保真、不造假点、扩展模态便宜。
- 缺点：训练前仍要 **resample 成稠密 batch**；工程复杂度高；容易 join 错位。

选型：

- **训练输入最终几乎总是稠密 batch**；稀疏更适合 **采集真源 / 分析层**。
- 实践：**稀疏采集 → 以控制频率或策略频率做稠密导出**；原包保留可追溯。

坑：

- 用「最近邻凑帧」把 10Hz 相机硬对齐到 50Hz 控制，造成动作与画面相位漂，BC 学到错误因果。
- 稀疏表里 null 与「传感器故障」语义混淆。

解决：

- 规定主时钟（如 20/30Hz policy rate）；只在容差内 associate；超差标记 invalid 并剔除。
- 增加 is_keyframe / valid_mask / source_rate 列，清洗可审计。

优化：

- Parquet 按 episode_index 排序 + 合理 row group；过滤 episode 时谓词下推，避免全表扫。
- 低维列用合适 dtype (float32、int16 量化关节角度需谨慎)。

Q4. 图像用 H.265 (HEVC) 相对 JPEG/PNG/H.264 的优势与代价？

答：

方案	磁盘	随机访问	训练 decode	画质/伪影
PNG	极大	好	快	无损
JPEG 逐帧	中	好	快	块效应
H.264 MP4	小	中（依赖 GOP）	中	运动模糊处易糊
H.265 MP4	更小	中偏难	更重	同码率通常优于 H.264

优势： 同画质下码率更低，多相机长时采集几乎刚需。

代价： 编码/解码 CPU 更重；B 帧与长 GOP 让「按 frame_index 精确抽帧」变慢且易 off-by-one。

坑：

1. 用 OpenCV/VideoCapture 当精确 random access：常常 seek 不准或代价很高（需从附近关键帧追到目标），不等于「每次必从第 0 帧解到尾」，但绝不是 JPEG 那种 0(1) 抽帧。
2. PTS/帧序号与 Parquet frame_index 差 1，整库 action 错位。
3. 采集用 yuv420，训练以为 RGB full range，颜色漂移。
4. 可变帧率 VFR 录像，长度对不上。

解决：

- 采集强制 CFR（恒定帧率），GOP 短一点（如 30-60）若需要较多随机访问；纯顺序训练可更长 GOP。
- 转换时写死 fps、总帧数校验：abs(n_frames_video - n_rows_episode) <= 容差，失败重编码。
- 索引侧存 video_path + frame_index，decode 用 PyAV/decord/torchvision，并做单元测试抽查 hash。

优化：

- 训练分辨率直接在转码时降到 backbone 输入（如 224/256/320），别每次 epoch 解 1080p。
- GPU decode (NVDEC) + 连续片段采样 + 预取队列；world model 重随机则转 HDF5 raw/JPEG 缓存。
- 多进程 DataLoader 注意「每 worker 独立 decoder」，避免句柄争用。

Q5. 设计一版「rosbag → LeRobot」转换流水线，关键步骤与校验点？

答：

1. **Inventory**: 列出 bag 内话题、频率、QoS、时间范围、缺帧率。
2. **选择 master clock**: 通常 policy_hz 或主相机。
3. **时间对齐**: message_filters 近似或离线 associate; 关节用插值, 图像用最近邻 (在阈值内)。
4. **坐标系与单位**: 统一 base/ee、弧度/角度、cmd vs measured。
5. **写 Parquet**: 一行一步; 写 episode_index/frame_index/timestamp。
6. **写视频**: 按相机拼帧编码 H.264/H.265; 记录 codec/fps。
7. **写 meta**: features shape、fps、robot_type、相机外参、任务文案。
8. **校验**: 帧数一致、无 NaN、动作限幅、时间单调、抽帧人工/自动质检。
9. **版本入库**: 数据集语义化版本 + 转换脚本 git hash。

坑: sim time / wall time 混用; TF 外推失败仍写入; 只转成功 demo 却没记 success 标签。

解决: 转换报告强制输出; CI 对小 bag 跑 golden test。

Q6. 数据清洗具体洗什么? 中高级要有「可量化标准」

答 (清洗清单):

1. **时间域**: 非单调 stamp、大间隙、回环、时钟跳变。
2. **同步域**: 跨模态对齐误差 > 阈值 (如图像-关节 > 20-30ms)。
3. **控制域**: 动作长期饱和、急停、碰撞力矩尖峰、人手拖动段。
4. **视觉域**: 过曝/全黑、花屏、丢相机、严重运动模糊。
5. **任务域**: 未达成功条件却标 success; 语言指令与轨迹不符。
6. **分布域**: 重复录同一条、场景单一、左右手/左右相机严重不平衡。

指标例:

- 坏 episode 率、平均对齐误差 p95、动作 clip 比例、成功标签准确率 (抽检)、类间/场景熵。

坑: 用「看起来顺滑」当清洗标准, 把恢复行为 (失败后再成功) 全删掉, 策略变脆。

解决: 分层保留——训练主集要干净成功; 另建 recovery/near_miss 子集做鲁棒性或不要混权。

优化: 清洗规则代码化 + 报表; 人工只审边界样本。

Q7. 动作与观测怎么归一化? 有哪些隐蔽坑?

答:

- 常见: 按数据集统计做 mean/std 或分位数缩放到 $[-1, 1]$ 。
- 关节角注意 wrap ($\pm\pi$); 夹爪开合可能是离散/二值。
- **绝对位姿 vs 相对增量**: 混用会让网络学崩。
- 多机器人混训必须 **embodiment 条件化** 或分头, 不能只靠同一套 normalize。

坑: 用训练集 mean/std 却在评估时重算; 或把失败轨迹一起进统计, 方差被噪声撑爆。

解决： stats 只来自 train split 成功子集；冻结进 meta；eval/infer 严格复用。

Q8. 「稀疏边稠密」若被理解成「边缘稀疏存储、中心稠密训练」，怎么答更高级？

答：

可以讲成 **两级存储策略**（面试加分）：

- **边（采集端/仓边）**：稀疏保真——按传感器原生频率写 bag/mcap 或分表 Parquet，图像走硬件/软件视频编码（H.264/H.265），尽量少做破坏性插值。
- **中（训练端）**：稠密对齐——按策略频率 resample，生成固定 schema 的训练视图；热点再 densify 成 HDF5/JPEG shard。

这句话把「格式争论」上升成 **数据平台架构**，比只背 HDF5 vs LeRobot 更像中高级。

2. 标注：位置估计 / BBox / 语义

Q9. 具身场景里为什么还要做检测框和语义？和纯 BC 的关系？

答：

纯模仿学习可以端到端吃像素，但标注仍有价值：

1. **数据筛选**：用「目标是否在视野/可抓取」过滤无效段。
2. **辅助损失 / 表征**：检测头、分割头稳定视觉骨干。
3. **语言对齐**：open-vocab grounding 把「红色杯子」落到区域。
4. **安全与部署**：在线检测做接近停止、人靠近降速。
5. **评测**：成功率之外的过程指标（是否对准目标）。

选型：任务若强依赖空间关系（抓取、跟随、避障），检测/分割 ROI 往往比盲目堆 demo 更划算。

Q10. BBox 表示与坐标系约定，工程上怎么统一？

答：

常见表示：

- xyxy 像素；xywh；cxcywh；以及 **归一化到 [0,1]** 的中心+宽高。

训练常用 YOLO 类归一化；ROS 可视化常用像素；策略若 crop ROI，要固定一种。

坑：letterbox 缩放后框没映射回原图；归一化宽高当像素用；左右相机外参搞反。

解决：单一模块负责 letterbox ↔ original 逆变换；单测固定图；约定文档写死字段（如 $cx, cy, w, h \in [0, 1]$ ）。

Q11. 讲清 YOLO 类实时检测在 ROS2 节点里的关键原理与优化点

答（原理）：

- Backbone+Neck 提多尺度特征；Head 回归框与类别（现代多为 anchor-free / 解耦头）。
- 后处理：置信度阈值 + NMS（或 NMS-free 头）。
- 输入预处理：BGR/RGB、letterbox、归一化与训练一致。

ROS2 工程点：

- 回调里不做重推理；独立推理线程 + 最新帧策略（丢旧帧保实时）。
- use_sim_time、QoS (sensor data)、与控制节点解耦话题。
- 输出结构化检测（label/conf/bbox）供过滤与控制。

坑：

- 训练 640、部署 320 且没重标校准。
- GPU 上下文在多节点重复初始化。
- NMS 后多目标选错（跟随场景跟错人/跟错柱）。

解决： 导出 ONNX/TensorRT 与预处理绑定版本号；目标选择加启发式（最大框、中心优先、类别白名单）。

优化： FP16/INT8、固定 shape、CUDA graph、ROI 追踪降频全图检测。

Q12. 语义分割 / 实例分割相对 bbox ，何时值得上？

答：

需求	bbox	mask
跟随/粗接近	通常够	过重
精确边缘抓取、线缆、不规则物体	不够	需要
可通行区域、地面污渍	弱	语义分割强
标注成本	低	高

选型： 先 bbox 闭环节能；mask 只加在误差来自「形状/接触」的环节。

坑： 分割模型 FPS 拖垮控制；小物体 IoU 好看但抓取仍失败（缺深度/力控）。

解决： 分割出 mask → 结合深度求抓取点；控制频率与感知频率分离。

Q13. 伪标注链路：Grounding + SAM 等怎么用于数据生产？坑在哪？

答：

典型弱监督：

1. 文本/类别 → GroundingDINO/OWL 出框；
2. 框 → SAM/SAM2 出 mask；
3. 可选投影到点云做 3D 粗框；
4. 人工抽检 + 置信度门控入库。

优势： 降标注成本，适合大规模回放 bag 补标。

坑：

- 开放词汇幻觉（把相似物体标成指令物体）。
- 时序不稳，相邻帧框跳，训练辅助头学噪声。
- 仿真可开挂用 GT，真机伪标域差被忽略。

解决：

- 置信度 + 时序平滑（轻量 IoU 匹配，不必上复杂 tracker 也可）。
- 关键任务词表，禁止过宽 prompt。
- 真机抽检比例提高；仿真与真机分库。

Q14. 从 2D bbox + 深度估计 3D 位置，误差链如何拆？

答：

误差来源串联：

1. 2D 框中心不是物理中心（遮挡、不对称物体）。
2. 深度噪声/透明/黑色物体失效。
3. 相机内参不准、手眼外参误差。
4. 时间不同步导致「框来自 t ，深度来自 $t+\Delta$ 」。
5. 目标点选在框中心 vs 底边中心（桌面物体）策略不同。

优化： 多帧融合、外参标定定期复查、用物体先验高度、力控/触觉闭环做最后一厘米。

Q15. 标注规范与质检，面试怎么讲才像带过团队？

答：

- 定义：正样本边界、遮挡规则、截断规则、忽略区。
- 双人抽检 κ 一致率；争议仲裁文档。
- 版本化：标签 schema v1/v2，旧数据 migration。
- 指标：类均 AP 不是唯一；要看 **任务成功相关的关键类**。

坑： 外包按 AP 优化狂标易检大物体，小零件漏标；模型 AP 升、抓取成功率不升。

解决： 任务导向的评测集与验收，不单看 COCO AP。

Q16. 检测抖动导致控制抖，调参还是上跟踪？

答（中高级取向）：

优先轻量：

1. 置信度门限、最小框面积、中心区域先验。
2. 对 cx, cy, w 做低通/一欧元滤波。
3. 控制侧死区与加速度限制。

仍不够再：IoU 匹配的 BYTE/BoT-SORT 等。

原则：能调参解决别上新层；新层默认关闭、可切换（面试官喜欢「可回退」）。

3. ROS2 采集同步与部署回环

Q17. ROS2 多传感器时间同步原理？Approximate vs Exact

答：

- **ExactTime**：时间戳完全相等才组包，真实系统几乎不可用。
- **ApproximateTime**：在队列里找时间最近的一组，超差则丢。
- 控制/关节可插值；图像通常不能「插出一帧」，只选最近。

坑：

- 各节点 stamp 用 `now()` 而不是采样时刻。
- 仿真 `use_sim_time:=false` 混进真值袋。
- 不同机器未做时钟同步（真机多机）。

解决：驱动层打传感器时间；仿真统一 `sim time`；多机 PTP/chrony；离线包用 stamp 重对齐。

Q18. 采集时 QoS、频率、丢帧怎么设计？

答：

- 图像：SensorDataQoS (Best Effort) 常见；落盘节点若跟不上要 **自降采样** 而不是堵驱动。
- 关节状态 Reliable + 足够 queue。
- 录制侧监控：实际 Hz、丢包计数、磁盘带宽。

坑：全部 Reliable + 大图把 DDS 打爆，看起来像「算法不稳」。

解决：先 profiling 网络与序列化；大图硬件压缩或录编码后 topic。

Q19. 部署回环：训练好的策略如何安全接回 ROS2？

答：

观测预处理(与训练一致)

- Policy 推理
- 动作反归一化 / 限幅
- 安全层 / 仲裁 (急停 > 遥控 > 策略)
- 底层驱动

同时：失败自动录包 (trigger bag) 回流清洗。

坑：

- 预处理不一致 (颜色、resize、图像增强仅训练有)。
- 训练是绝对关节目标，部署当增量速度发。
- 无 watchdog：策略节点挂了还在输出最后动作。

解决：同一份 preprocess 代码进训练与推理；动作空间单测；超时清零 + 安全门。

Q20. 在线与离线分布不一致 (sim2real / 策略漂移) 怎么谈？

答：

- 域随机化、轻微噪声注入、多场景采集。
- 观测加「任务相关」模态 (深度、状态) 减轻纯 RGB 过拟合。
- 在线收集 DAgger 式：遥操作纠正 → 混入训练集。
- 监控：动作分布 KL、触发安全次数、关键检测置信度。

4. 场景压测题 (区分中高级)

Q21. 100 小时、3 相机、磁盘报警，你怎么压成可训练的 LeRobot？

答要点：

1. 先统计每相机有效任务时段，裁掉 idle。
2. 转码 MP4 (H.264/H.265)，降到训练分辨率；CFR；校验帧数。
3. 低维进 Parquet shard (v3 多 episode/文件)。
4. 原 bag 冷存 Glacier/磁带；热集是 LeRobot。
5. 给出压缩比与随机抽检报告再删原图。

Q22. 训练 GPU 利用率只有 40%，你如何定位是 decode 问题？

答：

- nvidia-smi + DataLoader 耗时打点：time_to_batch vs forward。
- 换 JPEG cache / 预解码 HDF5 对比；若利用率立刻上去，即 IO/decode 瓶颈。
- 优化：短 GOP 或 cache、NVDEC、减少 worker 争用、连续采样、降低原分辨率。

Q23. 伪标 bbox 很多，但策略成功率不涨，怎么排查？

答：

- 伪标是否与任务成功相关，还是在标背景垃圾桶。
- 框抖动是否破坏辅助损失。
- 策略是否根本没用到 ROI/检测特征。
- 数据量/场景覆盖是否已是瓶颈。
- 做 ablation: 无标 BC vs 带标；看升的是 AP 还是 task success。

Q24. 多频率异构：图像 30Hz、关节 100Hz、语言 1 条/episode，如何进同一套训练 schema？

答：

- Episode 级字段放 meta (语言、任务 id)。
- 训练步以 20/30Hz 为行；关节插值；图像最近邻；语言 broadcast 到每步或用 encoder 条件一次。
- 保留 valid_mask；原稀疏真源可另存。

Q25. 若面试官问：你会从头自研格式吗？

答： 不会。优先 LeRobot 兼容 + 必要时训练缓存。自研格式只在：现有生态无法表达的关键模态、且团队有长期平台投入时。格式税极高。

5. 高频追问速答

Q： H.265 一定比 H.264 好？

A： 磁盘更好，decode 更贵；随机访问重的训练可能 H.264/短 GOP/JPEG 更优。

Q： Parquet 能存图像吗？

A： 能存 bytes，但大库不合适；应用列式管低维，视觉走视频或旁路文件。

Q： HDF5 并发写？

A： 多进程写同一文件易锁/损；按 episode 分文件再合并，或采集阶段别用 HDF5 当 WAL。

Q： 标注 AP 到多少够？

A： 没有绝对数；看下游任务敏感度。跟随/抓取看过程指标与成功率，而不是死磕 0.5 mAP。

Q： 同步误差多少可接受？

A： 与速度和任务有关；慢速操作几十 ms 往往可忍；高速接触要更严，并尽量硬件同步。

6. 可套用的项目经历模板（帮你「整理经历」）

下面三段是**可剪裁的叙事骨架**。请只保留你真实接触过的部分；没有的环节改成「课程/自学复现/阅读源码+小规模实验」。面试作假风险高，但「小规模真实闭环 + 清楚坑」完全够中高级讲述。

经历 A | 模仿学习数据管道（格式与清洗）——主打 Q1–Q8

一句话：

负责把 ROS2 采集的操作轨迹转成可训练数据集，对比 HDF5 与 LeRobot (Parquet + MP4) 方案，并落地清洗与质检。

背景：

多相机 + 关节/端控频率不一致，原始 bag 体积膨胀，训练 IO 打不满 GPU。

你的动作：

1. 定 master clock，离线 approximate associate，超差段作废。
2. 低维写 Parquet，图像 CFR 编码为 MP4 (H.264 或 H.265)；meta 固化 fps/shape/stats。
3. 清洗：时间乱序、动作饱和、全黑帧、失败误标 success。
4. 对比实验：纯视频随机采样 vs 预解码 JPEG/HDF5 缓存的 GPU 利用率与 epoch 时间。

结果（用你真实数字替换）：

- 磁盘下降约 X%（视频编码 + 裁 idle）。
- DataLoader 等待下降，GPU 利用率从 ~Y% 到 ~Z%。
- 坏 episode 占比从 A% 到 B%；策略成功率 +C%。

可讲的坑：

- 帧数与 Parquet 行数差 1 导致全身错位——用校验闸门拦住。
- 长 GOP 随机 seek 把训练拖死——对 world model/重随机任务转缓存。

面试口述 45 秒版：

「我做的不是单纯转格式，而是把采集保真和训练稠密视图拆开：边上视频编码+稀疏真源，训练前按策略频率对齐成 Parquet 行，并强制帧数/时间单调校验。后来发现随机时间窗任务下视频 decode 成瓶颈，就加了一层 HDF5/JPEG 热缓存，GPU 利用率才上去。」

经历 B | 检测伪标注服务数据闭环——主打 Q9–Q16

一句话：

搭建离线伪标注：检测/ grounding 出 bbox，必要时 SAM 出 mask，服务数据筛选与辅助监督，并控制时序抖动。

你的动作：

1. 统一 bbox 归一化约定与 letterbox 逆变换。
2. 置信度门控 + 轻量跨帧 IoU 匹配平滑。

- 3. 用「目标是否入画」过滤无效轨迹；抽样人工质检。
- 4. ablation：无标 BC vs 带检测辅助。

坑与解决：

- AP 升但任务不升 → 改任务导向抽检，去掉与任务无关类。
- 框抖传导到控制 → 先滤波与死区，再考虑跟踪。

口述 45 秒：

「标注对我不是交 AP，而是服务闭环：能过滤脏轨迹、能给表示学习稳一点的空间监督。我用门控+时序平滑控制伪标噪声，并且始终用任务成功率做验收，避免被 C0C0 指标带跑。」

经历 C | ROS2 采集与策略回灌安全——主打 Q17-Q20

一句话：

打通「策略推理节点 → 安全仲裁 → 底盘/手臂」，并做失败触发录包回流。

你的动作：

- 1. 推理与训练共享预处理。
- 2. 动作反归一化、限幅、超时清零。
- 3. 仲裁：急停 > 遥操作 > 策略。
- 4. 异常自动分段 bag，进入清洗队列。

坑： use_sim_time、stamp 用 now()、预处理不一致导致「离线指标好、线上乱」。

口述 45 秒：

「部署回环我抓两件事：训练-推理预处理一致性，以及安全门与失败回流。策略再强也不能绕过限幅和遥控接管；线上失败包要带着同步好的观测动作回来，否则数据飞轮转不起来。」

经历组合建议（按岗位）

岗位偏向	主讲	辅讲
数据平台 / 具身数据工程师	A	C
感知 / 标注算法	B	A
具身算法（BC/VLA）	A + 一点 B	C
ROS2 应用工程师偏具身	C	A

诚实表达句式（没有大厂项目时）

- 「这是我在开源栈（LeRobot / ROS2）上搭的最小闭环，数据量是 N 小时级，但问题类型和工业界同构。」
- 「大盘数字来自小规模对比实验，量级趋势可信，绝对值会随硬件变。」
- 「我没管过百人标注团队，但质检流程和任务导向验收我走过。」

这类表述中高级面试通常可接受，比编造「百万小时量产」安全。

7. 三日刷题计划

日	内容
D1	Q0-Q8 口述 + 画格式对比表；把经历 A 数字填空
D2	Q9-Q16 + 经历 B；准备 letterbox/NMS/伪标 ablation
D3	Q17-Q25 + 经历 C；模拟 10 分钟全链路演讲

8. 一页速查（临场前 5 分钟看）

面试官视角速记： 分层 > 名词；指标 > 感觉；可回退 > 叠层；任务成功率 > AP。

- **格式：** 分享/采集 → LeRobot；重随机访问 → HDF5 缓存；OpenX → RLDS。
- **Parquet：** 采集可稀疏保真；训练要稠密对齐；保留 valid_mask。
- **视频编码：** 省盘不省显存；CFR + 帧数校验；重随机就转码缓存。
- **清洗：** 时间、同步、饱和、花屏、错 success；规则代码化。
- **标注：** 为任务服务；伪标要门控+平滑；AP≠成功率。
- **ROS2：** ApproximateTime + 正确 stamp + sim time；部署要安全门与回流。
- **原则：** 能调参不叠层；格式跟生态；一切用任务指标验收。